

ĐỒ ÁN MÔN HỌC

ĐỒ ÁN IOT

HỆ THỐNG XE THÔNG MINH ĐIỀU KHIỂN KHÔNG DÂY ỨNG DỤNG IOT

Ngành: Robot và trí tuệ nhân tạo

Lớp: 22DRTA1

Giảng viên hướng dẫn : Ths. NGÔ AN THUYỀN

Sinh viên thực hiện:	MSSV:	Lớp:
Nguyễn Văn Đạt	2286300010	22DRTA1
Huỳnh Long	2286300028	22DRTA1
Nguyễn Chấn Huy	2286300020	22DRTA1

Tp.HCM, ngày ... tháng ... năm

ĐỒ ÁN MÔN HỌC

ĐỒ ÁN IOT

HỆ THỐNG XE THÔNG MINH ĐIỀU KHIỂN KHÔNG DÂY ỨNG DỤNG IOT

Ngành: Robot và trí tuệ nhân tạo

Lớp: 22DRTA1

Giảng viên hướng dẫn : Ths. NGÔ AN THUYỀN

Sinh viên thực hiện:	MSSV:	Lớp:
Nguyễn Văn Đạt	2286300010	22DRTA1
Huỳnh Long	2286300028	22DRTA1
Nguyễn Chấn Huy	2286300020	22DRTA1

Tp.HCM, ngày ... tháng ... năm

VIỆN KỸ THUẬT HUTECH**PHIẾU GIAO ĐỀ TÀI****TÊN MÔN HỌC : Đồ án IOT****NGÀNH: Robot và Trí tuệ nhân tạo****1. Họ và tên sinh viên/ nhóm sinh viên được giao đề tài (số số trong nhóm: 03):**

(1) Nguyễn Văn ĐạtMSSV: 2286300010..... Lớp: 22DRTA1

(2) Huỳnh LongMSSV: 2286300028..... Lớp: 22DRTA1

(3) Nguyễn Chấn HuyMSSV: 2286300020..... Lớp: 22DRTA1

2. Tên đề tài: Hệ thống xe thông minh điều khiển không dây ứng dụng IoT.....**3. Các dữ liệu ban đầu:**

- Kiến thức nền tảng về IoT, hệ thống điều khiển nhúng, và kết nối không dây.
- Các linh kiện phần cứng: vi điều khiển (ESP32/Arduino), cảm biến khoảng cách/IR/siêu âm, động cơ, pin, mạch điều khiển động cơ.
- Phần mềm: Dashboard/Webserver (trên máy tính hoặc điện thoại), công cụ lập trình (Arduino IDE, Python, hoặc tương đương).
- Tài liệu tham khảo từ các nghiên cứu trước về xe thông minh và ứng dụng IoT.

4. Nội dung nhiệm vụ :

- Nghiên cứu, thiết kế và chế tạo mô hình xe thông minh có khả năng điều khiển từ xa qua IoT.
- Tích hợp hệ thống cảm biến để thu thập dữ liệu (tốc độ, hướng di chuyển, mức pin, khoảng cách vật cản).
- Xây dựng giao diện Dashboard/Webserver để giám sát và điều khiển xe theo thời gian thực.
- Thiết kế PCB riêng cho khối điều khiển động cơ và khối cảm biến.
- Ứng dụng IoT để kết nối xe với server, lưu trữ dữ liệu hành trình và có thể mở rộng giám sát qua cloud.
- Nâng cao (tùy điều kiện):
 - Tích hợp AI để xe nhận diện line đường hoặc vật cản.
 - Tích hợp camera để quan sát trực tiếp từ xa.

5. Kết quả tối thiểu phải có:

- 1) Xe thông minh có thể điều khiển từ xa qua IoT (dashboard/Webserver).
- 2) Hệ thống giám sát thời gian thực trạng thái xe (tốc độ, hướng, mức pin, dữ liệu cảm biến).
- 3) Báo cáo đầy đủ: mô tả, thiết kế, thực nghiệm, kết quả đạt được, tài liệu tham khảo.

Ngày giao đề tài:/...../..... Ngày nộp báo cáo:/...../.....

TP. HCM, ngày ... tháng ... năm

Sinh viên thực hiện

(Ký và ghi rõ họ tên các thành viên)

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Nguyễn Văn Đạt

Huỳnh Long

Nguyễn Chấn Huy

Ngô An Thuyên

II. NỘI DUNG THỰC HIỆN

1. Tiền xử lý dữ liệu

Trong phạm vi đề tài “**Hệ thống xe thông minh điều khiển không dây ứng dụng IoT**”, dữ liệu được thu thập chủ yếu từ **encoder** (xung và tốc độ quay/RPM) và được **ESP32** gửi lên **broker HiveMQ** dưới dạng bản tin MQTT. Trên phía giám sát, **Node-RED** nhận dữ liệu theo thời gian thực để hiển thị biểu đồ và trạng thái vận hành. Do hệ thống tập trung vào giám sát thời gian thực, nhóm ưu tiên xử lý dữ liệu theo hướng đơn giản, dễ triển khai và phù hợp môi trường IoT. Cụ thể, dữ liệu RPM/đếm xung có thể được làm sạch bằng cách loại bỏ các bản tin lỗi (payload không đúng định dạng hoặc thiếu trường), đồng thời áp dụng lọc nhiễu nhẹ (ví dụ trung bình trượt với cửa sổ nhỏ) nhằm giảm dao động do rung động cơ khí hoặc nhiễu điện từ động cơ.

Trong trường hợp nhóm có nhu cầu lưu trữ dữ liệu để phân tích sau, Node-RED có thể ghi log dữ liệu theo timestamp ra file CSV. Khi đó, bước tiền xử lý có thể thực hiện bằng Python để chuẩn hóa dữ liệu: loại bỏ các dòng bị thiếu giá trị, lọc nhiễu bằng **moving average (window = 3)** hoặc lọc trung vị, và chuẩn hóa thang đo (scale) nếu cần phục vụ thống kê/đánh giá. Tuy nhiên, việc chuẩn hóa và phân tích sâu chỉ thực sự cần thiết khi nhóm triển khai các bài toán dự đoán hoặc tối ưu điều khiển ở mức nâng cao.

2. Xây dựng mô hình AI (tùy chọn – chỉ triển khai khi mở rộng)

Ở phiên bản hiện tại, hệ thống đã đáp ứng tốt yêu cầu điều khiển và giám sát; do đó, **AI không bắt buộc** để hoàn thành mục tiêu tối thiểu. Tuy nhiên, nếu mở rộng theo hướng nâng cao, AI có thể được sử dụng để hỗ trợ **cảnh báo vận hành** hoặc **hỗ trợ điều khiển** dựa trên dữ liệu thu thập. Với dữ liệu encoder, một hướng khả thi là xây dựng mô hình phân loại trạng thái hoạt động như: xe chạy ổn định, xe bị kẹt bánh/ma sát lớn, hoặc xe bị trượt (RPM tăng nhưng xe không di chuyển tương ứng). Khi đó, dữ liệu đầu vào có thể gồm RPM trái/phải, độ lệch RPM giữa hai bánh và các đặc trưng thống kê theo cửa sổ thời gian ngắn.

Ngoài ra, nếu nhóm sử dụng camera/Jetson Nano, AI phù hợp nhất là **xử lý ảnh** (nhận diện vật cản, bám line) thay vì chỉ dựa trên encoder. Đây là hướng mở rộng có ý nghĩa ứng dụng rõ rệt với xe thông minh. Các mô hình nhẹ (ví dụ phát hiện line bằng OpenCV hoặc nhận diện vật cản bằng mô hình nhỏ) có thể chạy tại thiết bị biên để đưa ra cảnh báo hoặc đề

xuất hành động an toàn (giảm tốc/dừng), giúp hệ thống tiệm cận hơn với robot di động trong thực tế.

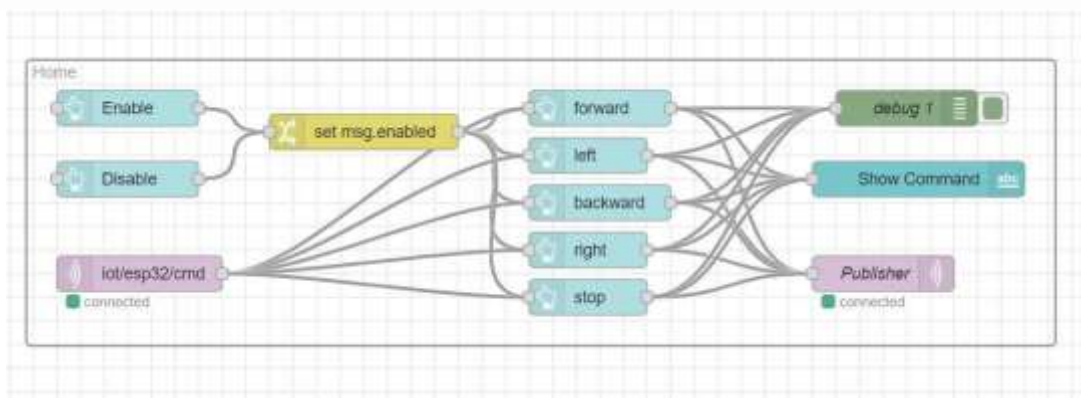
3. Tích hợp AI vào hệ thống (*tùy chọn – chỉ triển khai khi mở rộng*)

Nếu triển khai AI, phân tích hợp nên tách khỏi ESP32 để đảm bảo ổn định và tránh quá tải tài nguyên nhúng. Một cách tiếp cận phù hợp là triển khai mô-đun AI trên máy tính/Jetson hoặc server cục bộ dưới dạng **API** (có thể dùng Flask/FastAPI). Node-RED sẽ đóng vai trò trung gian: subscribe dữ liệu từ MQTT, gửi dữ liệu cần thiết sang API để dự đoán, sau đó nhận kết quả và hiển thị lên Dashboard. Nhờ đó, hệ thống vẫn giữ kiến trúc IoT rõ ràng: ESP32 thu thập dữ liệu → HiveMQ truyền dữ liệu → Node-RED xử lý/hiển thị → AI xử lý nâng cao khi cần.

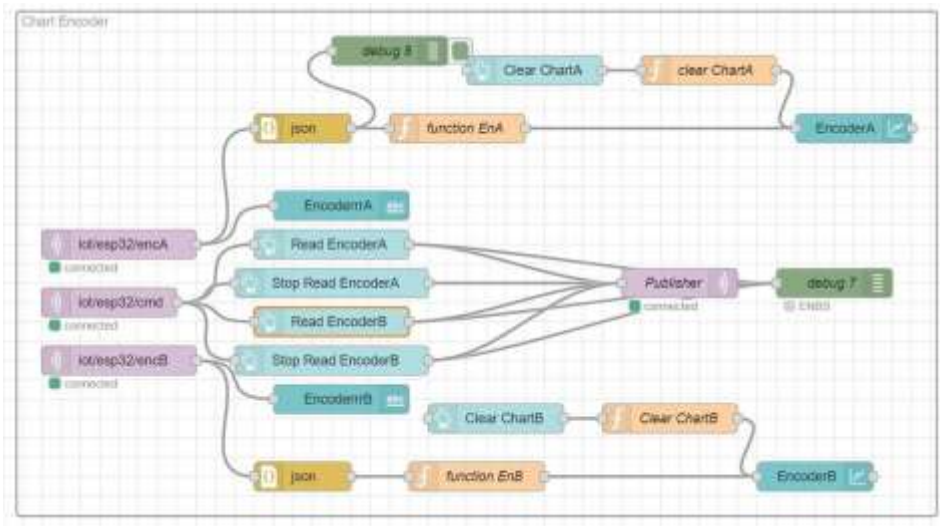
Trên giao diện Dashboard, kết quả AI có thể được biểu diễn dưới dạng **cảnh báo trực quan**: đổi màu trạng thái, hiển thị nhãn “Bình thường/Cảnh báo”, hoặc bật thông báo khi phát hiện bất thường như lệch tốc độ hai bánh lớn hoặc tình trạng kẹt bánh. Với trường hợp AI xử lý ảnh, kết quả có thể là cảnh báo “Có vật cản phía trước” hoặc “Lệch line”, đồng thời hệ thống có thể kích hoạt chế độ an toàn như tự dừng nếu người dùng bật chế độ hỗ trợ. Cách tích hợp này giúp tăng giá trị ứng dụng mà vẫn đảm bảo hệ thống chính (điều khiển IoT + giám sát) hoạt động ổn định.

III. KẾT QUẢ THU ĐƯỢC

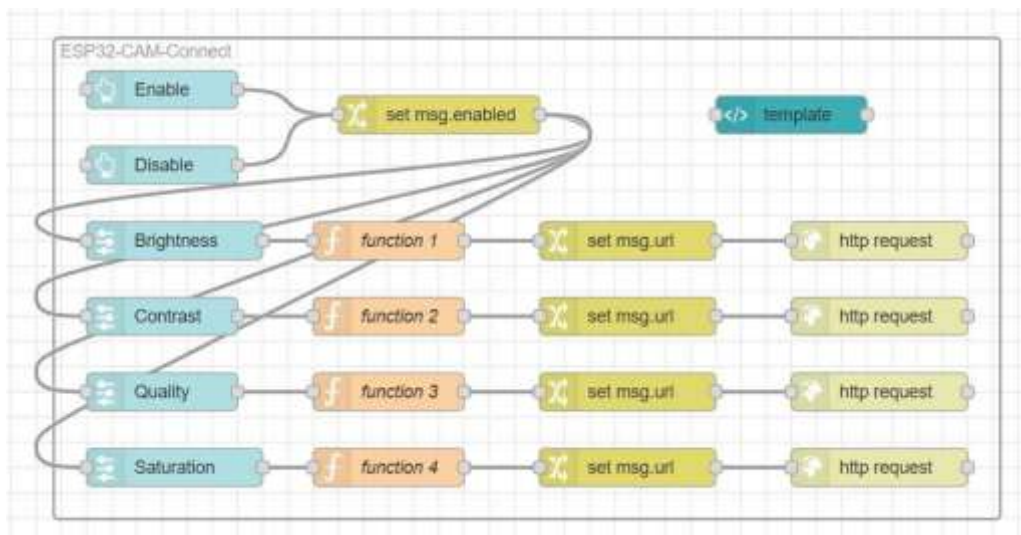
Ảnh giao diện dashboard



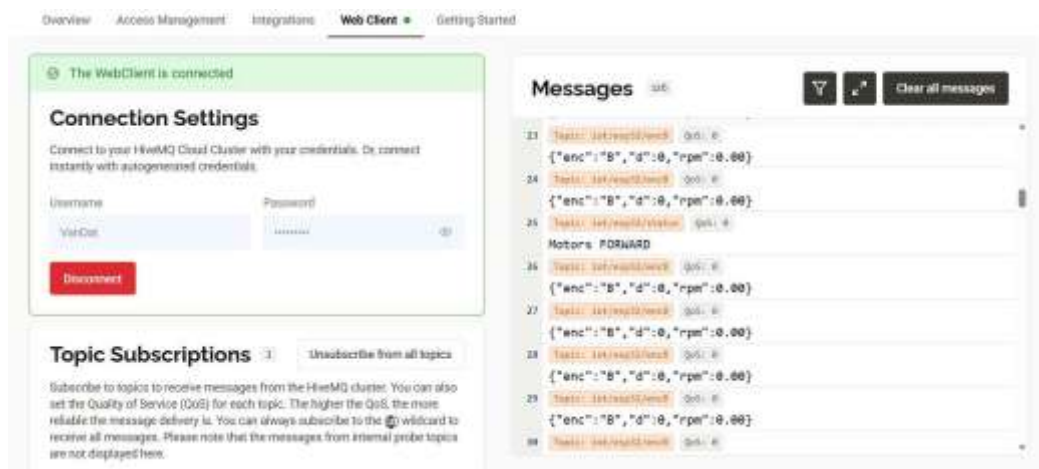
Hình 4.3 Thiết kế giao diện điều khiển cho hệ thống trên Node-red



Hình 4.4 Thiết kế giao diện giám sát encoder cho hệ thống trên Node-red



Hình 4.5 Thiết kế giao diện tùy chỉnh camera cho hệ thống trên Node-red



Hình 4.7 Web Client của hệ thống trên HiveMQ



Hình 5.1 Giao diện điều khiển của hệ thống trên Dashboard



Hình 5.2 Giao diện giám sát encoder của hệ thống trên Dashboard



Hình 5.3 Giao diện điều chỉnh camera của hệ thống trên Dashboard

Trích đoạn mã code:

```
- #include <Arduino.h>
- #include <WiFi.h>
- #include <WiFiClientSecure.h>
- #include <PubSubClient.h>
-
- // ===== L298N =====
- const int IN1 = 26;
- const int IN2 = 27;
- const int ENA_PIN = 25;
-
- const int IN3 = 12;
- const int IN4 = 13;
- const int ENB_PIN = 14;
-
- // ===== ENCODER LEFT (A) =====
- #define ENCL_C1 35
- #define ENCL_C2 34
-
- // ===== ENCODER RIGHT (B) =====
- #define ENCR_C1 32
- #define ENCR_C2 33
-
- // ===== PWM =====
- const int freq = 5000;
- const int ledChannelA = 0;
- const int ledChannelB = 1;
- const int resolution = 8; // 0..255
-
- // ===== Encoder counts =====
- volatile long encoderLeftCount = 0;
- volatile long encoderRightCount = 0;
- portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
-
- // ===== PPR hiệu dụng =====
- const float PPR_EFF = 40.0f;
-
- // ===== Latest encoder values (cập nhật mỗi 100ms) =====
- volatile long lastDL = 0, lastDR = 0;
- volatile float lastRpmL = 0.0f, lastRpmR = 0.0f;
-
- // ===== WiFi config =====
- const char* WIFI_SSID = "ChanHuy";
- const char* WIFI_PASSWORD = "12345671";
-
- // ===== MQTT (HiveMQ Cloud) config =====
- const char* mqtt_server =
- "43bada278be7468bb212609d24e98ec7.s1.eu.hivemq.cloud";
- const int mqtt_port = 8883;
```

```

- const char* mqtt_user      = "VanDat";
- const char* mqtt_pass      = "RobotCar1";
- const char* mqtt_client_id = "esp32_iot_xe";
- const char* mqtt_topic_sub = "iot/esp32/cmd";
- const char* mqtt_topic_pub = "iot/esp32/status";
-
- // encoder topics
- const char* mqtt_topic_encA = "iot/esp32/encA";
- const char* mqtt_topic_encB = "iot/esp32/encB";
-
- // ===== MQTT objects =====
- WiFiClientSecure secureClient;
- PubSubClient mqtt(secureClient);
-
- // ===== Robot control =====
- uint8_t speedA = 255;
- uint8_t speedB = 255;
-
- enum MoveCmd { STOP, FORWARD, BACKWARD, LEFT, RIGHT };
- volatile MoveCmd currentCmd = STOP;
-
- // ===== Encoder streaming flags =====
- volatile bool streamEncA = false;
- volatile bool streamEncB = false;
-
- // ===== Publish interval for encoder (ms) =====
- const uint32_t ENC_PUB_MS = 200;
-
- // ----- Forward declarations -----
- void publishStatus(const char* msg);
- void publishEncoderA();
- void publishEncoderB();
-
- // ===== INTERRUPT =====
- void IRAM_ATTR encoderLeftISR() {
-     int a = digitalRead(ENCL_C1);
-     int b = digitalRead(ENCL_C2);
-     portENTER_CRITICAL_ISR(&mux);
-     encoderLeftCount += (a == b) ? 1 : -1;
-     portEXIT_CRITICAL_ISR(&mux);
- }
-
- void IRAM_ATTR encoderRightISR() {
-     int a = digitalRead(ENCR_C1);
-     int b = digitalRead(ENCR_C2);
-     portENTER_CRITICAL_ISR(&mux);
-     encoderRightCount += (a == b) ? 1 : -1;
-     portEXIT_CRITICAL_ISR(&mux);
- }

```

```

- // ===== Status helper =====
- void publishStatus(const char* msg) {
-     Serial.println(msg);
-     if (mqtt.connected()) mqtt.publish(mqtt_topic_pub, msg, true);
- }
-
- // ===== Motor control =====
- void motorsStop() {
-     digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
-     digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
-     ledcWrite(ledChannelA, 0);
-     ledcWrite(ledChannelB, 0);
-     publishStatus("Motors STOP");
- }
-
- void motorsForward() {
-     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
-     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
-     ledcWrite(ledChannelA, speedA);
-     ledcWrite(ledChannelB, speedB);
-     publishStatus("Motors FORWARD");
- }
-
- void motorsBackward() {
-     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
-     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
-     ledcWrite(ledChannelA, speedA);
-     ledcWrite(ledChannelB, speedB);
-     publishStatus("Motors BACKWARD");
- }
-
- void motorsLeft() { // trái lùi, phải tiến
-     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
-     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
-     ledcWrite(ledChannelA, speedA);
-     ledcWrite(ledChannelB, speedB);
-     publishStatus("Motors LEFT");
- }
-
- void motorsRight() { // trái tiến, phải lùi
-     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
-     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
-     ledcWrite(ledChannelA, speedA);
-     ledcWrite(ledChannelB, speedB);
-     publishStatus("Motors RIGHT");
- }
-
- void applyCommand(MoveCmd cmd) {
-     switch (cmd) {
-         case FORWARD: motorsForward(); break;
-         case BACKWARD: motorsBackward(); break;
-         case LEFT:     motorsLeft();     break;
-         case RIGHT:    motorsRight();    break;
-         default:       motorsStop();     break;
-     }
- }

```

```

-     }
- }
-
- // ===== WiFi helper =====
- void connectWiFi() {
-     if (WiFi.status() == WL_CONNECTED) return;
-
-     Serial.print("Connecting WiFi: ");
-     Serial.println(WIFI_SSID);
-
-     WiFi.mode(WIFI_STA);
-     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
-
-     unsigned long t0 = millis();
-     while (WiFi.status() != WL_CONNECTED) {
-         delay(300);
-         Serial.print(".");
-         if (millis() - t0 > 20000) {
-             Serial.println("\nWiFi timeout -> retry");
-             WiFi.disconnect(true);
-             delay(500);
-             WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
-             t0 = millis();
-         }
-     }
-
-     Serial.println("\nWiFi connected!");
-     Serial.print("IP: ");
-     Serial.println(WiFi.localIP());
- }
-
- // ===== Encoder publish helpers =====
- void publishEncoderA() {
-     char msg[128];
-     long dl; float rpm;
-     portENTER_CRITICAL(&mux);
-     dl = lastDL; rpm = lastRpmL;
-     portEXIT_CRITICAL(&mux);
-
-     snprintf(msg, sizeof(msg),
-              "{\"enc\":\"A\",\"d\":%ld,\"rpm\":%.2f}", dl, rpm);
-
-     if (mqtt.connected()) mqtt.publish(mqtt_topic_encA, msg, false);
- }
-
- void publishEncoderB() {
-     char msg[128];
-     long dr; float rpm;
-     portENTER_CRITICAL(&mux);
-     dr = lastDR; rpm = lastRpmR;

```

```

- portEXIT_CRITICAL(&mux);
-
- snprintf(msg, sizeof(msg),
-          "{ \"enc\": \"B\", \"d\": %ld, \"rpm\": %.2f }", dr, rpm);
-
- if (mqtt.connected()) mqtt.publish(mqtt_topic_encB, msg, false);
- }
-
- // ===== MQTT helpers =====
- static inline void toUpperInPlace(char* s) {
-     for (; *s; ++s) *s = (char)toupper((unsigned char)*s);
- }
-
- void onMqttMessage(char* topic, byte* payload, unsigned int length) {
-     if (length == 0) return;
-
-     static char buf[64];
-     unsigned int n = (length < sizeof(buf) - 1) ? length : (sizeof(buf) -
1);
-     memcpy(buf, payload, n);
-     buf[n] = '\0';
-
-     while (n > 0 && (buf[n-1] == '\n' || buf[n-1] == '\r' || buf[n-1] == '
' || buf[n-1] == '\t')) {
-         buf[n-1] = '\0';
-         n--;
-     }
-     toUpperInPlace(buf);
-
-     Serial.print("MQTT IN [");
-     Serial.print(topic);
-     Serial.print("] : ");
-     Serial.println(buf);
-
-     // ---- encoder stream control ----
-     if (strcmp(buf, "ENA") == 0) {
-         streamEncA = true;
-         publishStatus("ENC A STREAM ON");
-         return;
-     }
-     if (strcmp(buf, "ENAS") == 0) {
-         streamEncA = false;
-         publishStatus("ENC A STREAM OFF");
-         return;
-     }
-     if (strcmp(buf, "ENB") == 0) {
-         streamEncB = true;
-         publishStatus("ENC B STREAM ON");
-         return;
-     }
- }

```

```

-   if (strcmp(buf, "ENBS") == 0) {
-       streamEncB = false;
-       publishStatus("ENC B STREAM OFF");
-       return;
-   }
-
-   // ---- movement commands (H/B/L/R/S) ----
-   MoveCmd next = STOP;
-   char c = buf[0];
-
-   if (c == 'H') next = FORWARD;
-   else if (c == 'B') next = BACKWARD;
-   else if (c == 'L') next = LEFT;
-   else if (c == 'R') next = RIGHT;
-   else if (c == 'S') next = STOP;
-   else {
-       publishStatus("Unknown CMD (H/B/L/R/S, ENA/ENAS, ENB/ENBS)");
-       return;
-   }
-
-   currentCmd = next;
-   applyCommand(currentCmd);
- }
-
- void connectMQTT() {
-     while (!mqtt.connected()) {
-         Serial.print("Connecting MQTT...");
-
-         const char* willTopic = mqtt_topic_pub;
-         const char* willMsg    = "offline";
-
-         bool ok = mqtt.connect(
-             mqtt_client_id,
-             mqtt_user,
-             mqtt_pass,
-             willTopic, 0, true, willMsg
-         );
-
-         if (ok) {
-             Serial.println("OK");
-             mqtt.publish(mqtt_topic_pub, "online", true);
-
-             if (mqtt.subscribe(mqtt_topic_sub)) {
-                 Serial.print("Subscribed: ");
-                 Serial.println(mqtt_topic_sub);
-             } else {
-                 Serial.println("Subscribe FAIL");
-             }
-         } else {
-             Serial.print("FAILED, state=");

```

```

-     Serial.print(mqtt.state());
-     Serial.println(" -> retry in 2s");
-     delay(2000);
- }
- }
- }

- // ===== Encoder compute task (mỗi 100ms) =====
- void updateEncoderEvery100ms() {
-     static uint32_t t0 = 0;
-     const uint32_t dt_ms = 100;

-     if (millis() - t0 < dt_ms) return;
-     t0 += dt_ms;

-     long dL, dR;
-     portENTER_CRITICAL(&mux);
-     dL = encoderLeftCount;  encoderLeftCount = 0;
-     dR = encoderRightCount; encoderRightCount = 0;
-     portEXIT_CRITICAL(&mux);

-     float dt_s = dt_ms / 1000.0f;
-     float rpmL = ((float)dL / PPR_EFF) * (60.0f / dt_s);
-     float rpmR = ((float)dR / PPR_EFF) * (60.0f / dt_s);

-     portENTER_CRITICAL(&mux);
-     lastDL = dL;  lastDR = dR;
-     lastRpmL = rpmL; lastRpmR = rpmR;
-     portEXIT_CRITICAL(&mux);
- }

- // ===== Encoder publish task (mỗi ENC_PUB_MS nếu stream ON) =====
- void streamEncoderIfEnabled() {
-     static uint32_t tPubA = 0, tPubB = 0;
-     uint32_t now = millis();

-     if (streamEncA && (now - tPubA >= ENC_PUB_MS)) {
-         tPubA = now;
-         publishEncoderA();
-     }
-     if (streamEncB && (now - tPubB >= ENC_PUB_MS)) {
-         tPubB = now;
-         publishEncoderB();
-     }
- }

- void setup() {
-     Serial.begin(115200);
-     delay(200);
- }

```

```

- // Motor pins
- pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
- pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
-
- // Encoder pins
- pinMode(ENCL_C1, INPUT_PULLUP);
- pinMode(ENCL_C2, INPUT_PULLUP);
- pinMode(ENCR_C1, INPUT_PULLUP);
- pinMode(ENCR_C2, INPUT_PULLUP);
-
- attachInterrupt(digitalPinToInterrupt(ENCL_C1), encoderLeftISR,
CHANGE);
- attachInterrupt(digitalPinToInterrupt(ENCR_C1), encoderRightISR,
CHANGE);
-
- // PWM
- ledcSetup(ledChannelA, freq, resolution);
- ledcAttachPin(ENA_PIN, ledChannelA);
-
- ledcSetup(ledChannelB, freq, resolution);
- ledcAttachPin(ENB_PIN, ledChannelB);
-
- motorsStop();
-
- connectWiFi();
-
- // TLS demo mode
- secureClient.setInsecure();
- secureClient.setHandshakeTimeout(30);
-
- mqtt.setServer(mqtt_server, mqtt_port);
- mqtt.setCallback(onMqttMessage);
-
- connectMQTT();
-
- Serial.println("RobotCar MQTT + Encoder stream ready!");
- Serial.println("Move: H/B/L/R/S");
- Serial.println("Encoder stream: ENA / ENAS, ENB / ENBS");
- }
-
- void loop() {
-   connectWiFi();
-   if (!mqtt.connected()) connectMQTT();
-   mqtt.loop();
-
-   updateEncoderEvery100ms();
-   streamEncoderIfEnabled();
- }
-

```

-
